



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – ВАРНА

Факултет по изчислителна техника и автоматизация
Катедра „Софтуерни и интернет технологии“

Курсов проект

по дисциплината: **Обектно-ориентирано
програмиране – 1 част**

на тема: **„НЕДЕТЕРМИНИРАН КРАЕН
АВТОМАТ“**

Изготвил:

Йоан Райчев Ф№22621716

Специалност: **СИТ**

Група: **1Б**

Съдържание

Глава 1. Увод.....	3
1.1. Описание и идея на проекта	3
1.2. Цел и задачи на разработката	3
1.3. Структура на документацията	4
Глава 2. Преглед на предметната област.....	4
2.1. Основни дефиниции, концепции и алгоритми.....	4
2.2. Дефиниране на проблеми и сложност на поставената задача.....	5
2.3. Подходи и методи за решаване на поставените проблеми	5
2.4. Потребителски и функционални изисквания.....	6
Глава 3. Проектиране.....	7
3.1. Обща структура на проекта	7
3.2 Архитектура на системата	7
Глава 4. Реализация и тестване.....	10
4.1. Тестване	55
Глава 5. Заключение	56
5.1. Обобщение на изпълнението на началните цели.....	56
5.2. Насоки за бъдещо развитие и усъвършенстване	57

Глава 1. Увод

1.1. Описание и идея на проекта

Проектът представлява софтуерно приложение, разработено на Java, което поддържа работа с недетерминирани крайни автомати (NFA) с ϵ -преходи. Автоматите работят върху азбука, състояща се от малки латински букви (a-z) и цифри (0-9).

- Приложението предоставя команден ред (CLI), чрез който потребителят може да:
- Зарежда и записва автомати във файлове (JSON формат)
- Извършва операции върху автомати (обединение, конкатенация, звезда)
- Проверява свойства на автоматите (детерминираност, празен език, краен език)
- Детерминира NFA до DFA
- Разпознава думи от езика на автомата
- Създава автомати от регулярни изрази

Всеки автомат, зареден от файл или създаден чрез операция, получава уникален идентификатор (ID), който се използва за позоваване в командите.

1.2. Цел и задачи на разработката

Цел на проекта: Да се създаде функционална програма за работа с недетерминирани крайни автомати, която да служи като учебно помагало за разбиране на теорията на автоматите.

Задачи за реализация:

- Разработване на JSON формат за сериализация на автомати
- Реализиране на команден интерфейс с поддръжка на командите: open, close, save, saveas, help, exit
- Имплементиране на операции за работа с автомати: list, print, recognize, deterministic, empty, union, concat, star, determinize, reg, finite
- Обработка на грешки при невалиден вход от потребителя
- Изготвяне на техническа документация

1.3. Структура на документацията

Документацията е организирана в пет глави. Глава 1 представя идеята, целта и задачите на проекта. Глава 2 разглежда основните понятия от теорията на автоматите и изискванията към програмата. Глава 3 описва архитектурата и проектирането на системата. Глава 4 представя

реализацията на ключовите класове и тестовите сценарии. Глава 5 обобщава постигнатите резултати и дава насоки за бъдещо развитие.

Глава 2. Преглед на предметната област

2.1. Основни дефиниции, концепции и алгоритми

Краен автомат (Finite Automaton) е математически модел, който се състои от краен брой състояния и преходи между тях. Той разпознава (приема) думи, съставени от символи на дадена азбука.

Недетерминиран краен автомат (NFA) допуска:

- Множество преходи от едно състояние с един и същ символ
- ϵ -преходи (преходи, които не консумират символ)

Детерминиран краен автомат (DFA) има точно един преход от всяко състояние за всеки символ от азбуката и няма ϵ -преходи.

Регулярен израз е текстово описание на език. Теоремата на Клини твърди, че регулярните изрази и крайните автомати са еквивалентни – за всеки регулярен израз съществува автомат, който разпознава същия език.

Използвани алгоритми:

- ϵ -closure – намира всички състояния, достижими от дадено състояние само чрез ϵ -преходи
- Subset construction – алгоритъм за детерминиране на NFA до DFA
- Thompson construction – алгоритъм за преобразуване на регулярен израз в NFA

2.2. Дефиниране на проблеми и сложност на поставената задача

Реализацията на програма за работа с недетерминирани крайни автомати поставя няколко основни проблема:

- Сериализация и десериализация – разработване на JSON формат, който да съхранява състояния, преходи (включително ϵ -преходи), начално и крайни състояния.
- Работа с ϵ -преходи – те добавят сложност при разпознаване на думи (ϵ -closure), детерминиране и операциите union, concat, star.
- Сложност на алгоритмите:

- recognize – $O(|\text{word}| \cdot |Q|^2)$
 - determinize – $O(2^h)$ (експоненциално)
 - union, concat, star – $O(|Q_1| + |Q_2|)$
 - reg – $O(|\text{regex}|)$
- Управление на автоматите – централизиран регистър с уникални ID-та и възможност за превключване между автоматите.
 - Команден интерфейс – парсане на команди, валидация на аргументи и обработка на потребителски грешки.
 - Точност на операциите – при union, concat, star не трябва да се променят оригиналните автомати, което изисква дълбоко копиране и префиксиране на имената на състоянията.

2.3. Подходи и методи за решаване на поставените проблеми

За решаване на проблемите, описани в 2.2, са използвани следните подходи:

- Сериализация чрез JSON – използвана е библиотеката Gson, която позволява лесно преобразуване на автоматите във файлов формат.
- Command Pattern – всяка команда от CLI е отделен клас, имплементиращ общ интерфейс. Фабрика (CommandFactory) създава съответния команден обект според въведената команда.

Алгоритми:

- ϵ -closure – за обработка на ϵ -преходи при разпознаване на думи
- Subset construction – за детерминиране на NFA до DFA
- Thompson construction – за преобразуване на регулярни изрази в NFA

Дер сору с префикси – при операциите union, concat и star се създават дълбоки копия с префикси ($A_$, $B_$), за да не се променят оригиналните автомати и да се избегнат колизии при еднакви имена на състояния.

Обработка на грешки – програмата проверява входните данни и извежда конкретни съобщения при липсващи или невалидни аргументи, без да прекъсва изпълнението си.

2.4. Потребителски и функционални изисквания

Функционални изисквания – основни команди:

Команда	Описание
open <file>	Отваря файл с автомат

close	Затваря текущия автомат
save	Записва промените в текущия файл
saveas <file>	Записва автомата в нов файл
help	Показва списък с всички команди
exit	Излиза от програмата

Специфични операции за работа с автомати:

Команда	Описание
list	Извежда списък с ID-тата на всички заредени автомати
print	Извежда всички преходи в текущия автомат
recognize <word>	Проверява дали дума се приема от автомата
deterministic	Проверява дали автоматът е детерминиран
determinize	Превръща NFA в DFA (мутатор)
empty	Проверява дали езикът на автомата е празен
union <id>	Обединение на текущия автомат с друг автомат
concat <id>	Конкатенация на текущия автомат с друг автомат
star	Позитивна обвивка (Kleene star) на текущия автомат
reg <regex>	Създава автомат от регулярен израз
finite	Проверява дали езикът на автомата е краен
select <id>	Смяна на текущия автомат

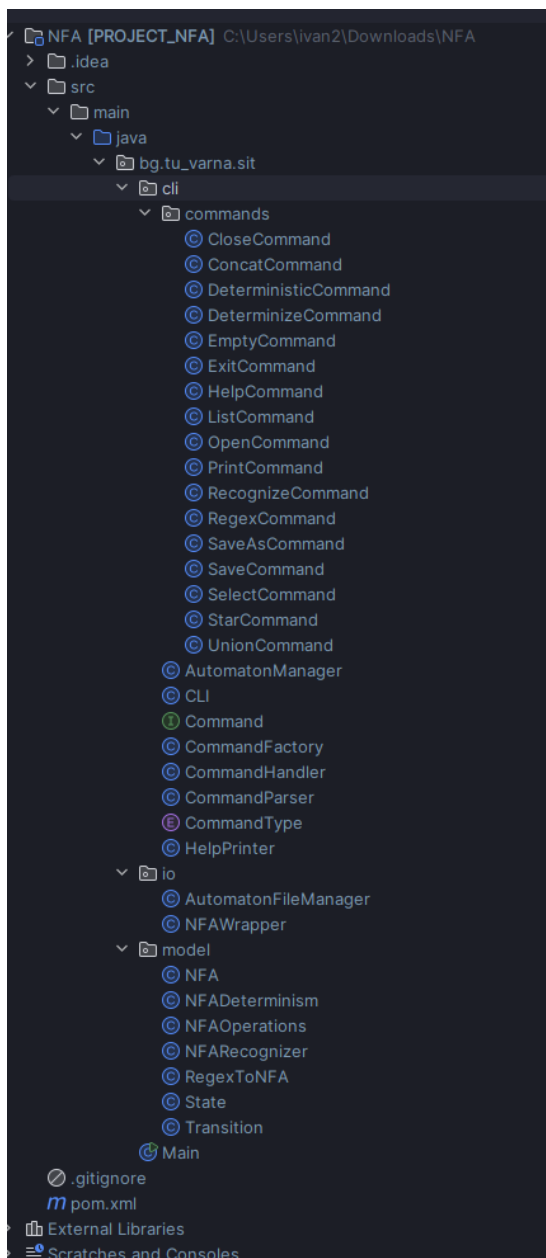
Нефункционални изисквания:

- Стабилност – програмата не трябва да "гърми" при невалиден вход
- Четимост на кода – Използване на смислени имена на класове и методи
- Разширяемост – лесно добавяне на нови команди чрез Command Pattern
- Документация – наличие на техническа документация и README файл

Глава 3. Проектиране

3.1. Обща структура на проекта

Проектът е организиран в следните пакети:



3.2 Архитектура на системата

Проектът е разделен на четири основни пакета:

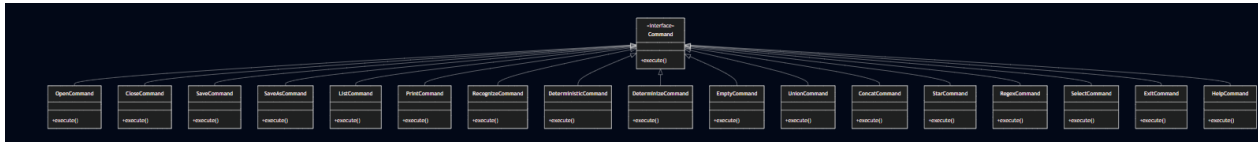
1. Пакет `bg.tu_varna.sit`

- `Main.java` – входна точка на програмата, стартира CLI

2. Пакет `cli` – команден интерфейс

- `CLI.java` – главен цикъл за четене на команди
- `Command.java` – интерфейс за всички команди
- `CommandFactory.java` – фабрика, която създава команди по име
- `CommandHandler.java` – обработва изпълнението на командите
- `CommandParser.java` – парсва входния ред от потребителя
- `CommandType.java` – изброен тип с всички поддържани команди

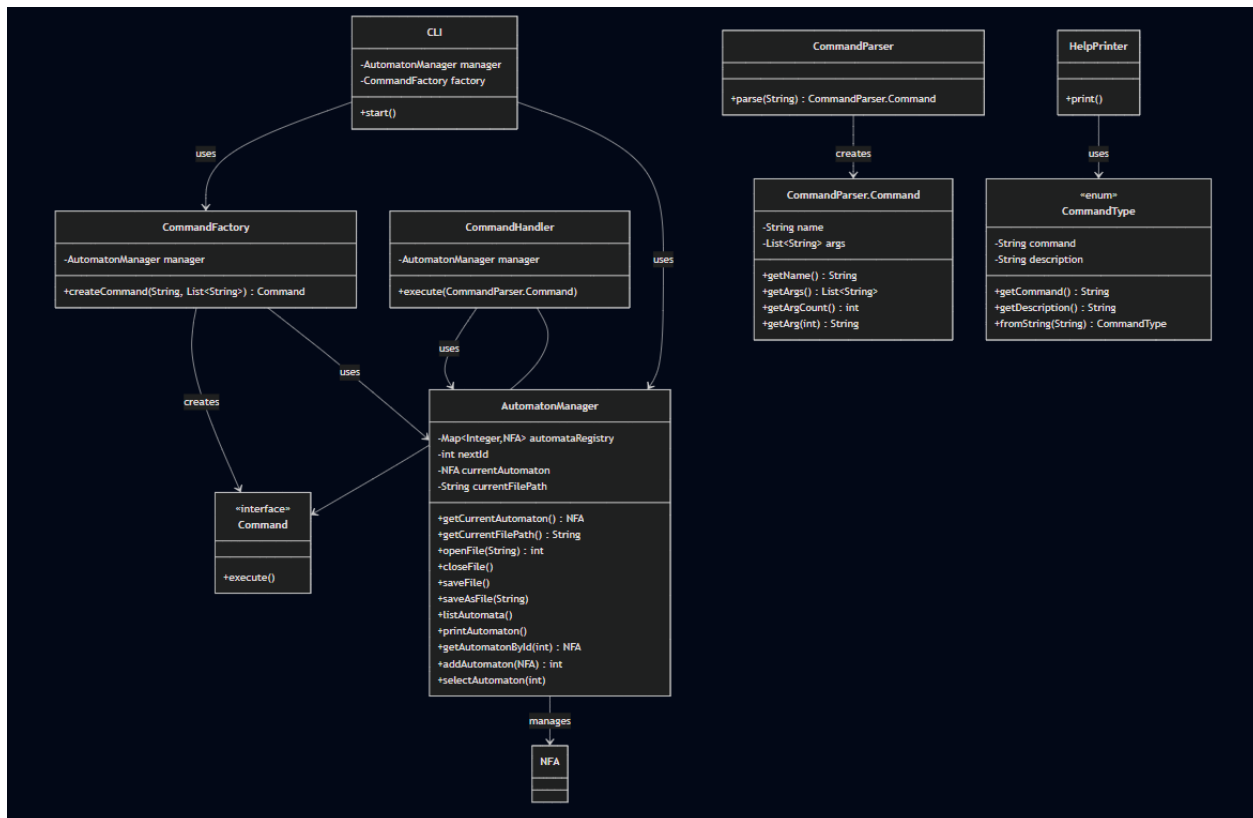
- AutomatonManager.java – Singleton клас, управляващ регистъра с автомати
- HelpPrinter.java – извежда помощното меню



Пакет cli.commands – съдържа всички конкретни команди:

- OpenCommand, CloseCommand, SaveCommand, SaveAsCommand
- ListCommand, PrintCommand, RecognizeCommand
- DeterministicCommand, DeterminizeCommand, EmptyCommand
- UnionCommand, ConcatCommand, StarCommand, RegexCommand
- SelectCommand, ExitCommand, HelpCommand

3. Пакет model – модели на автоматите

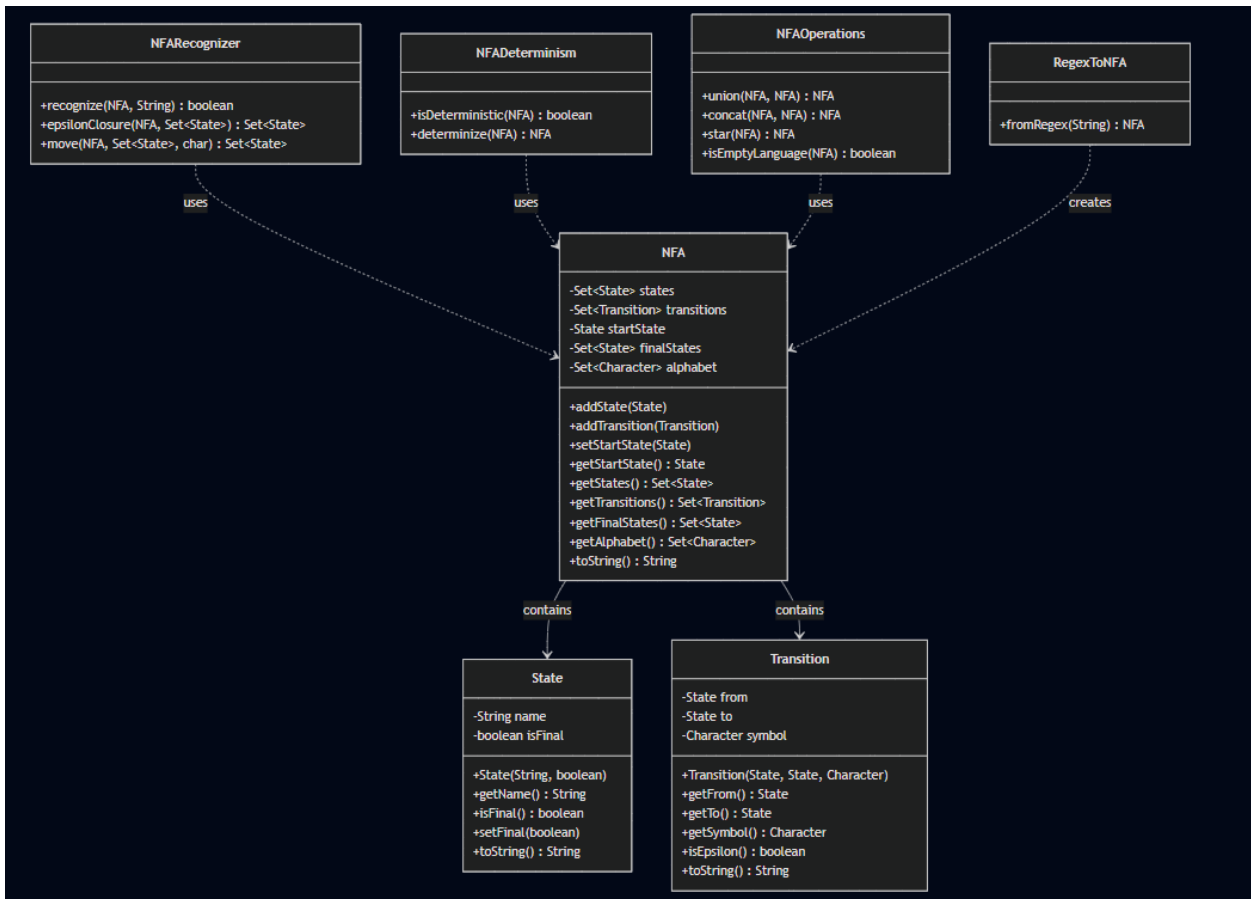


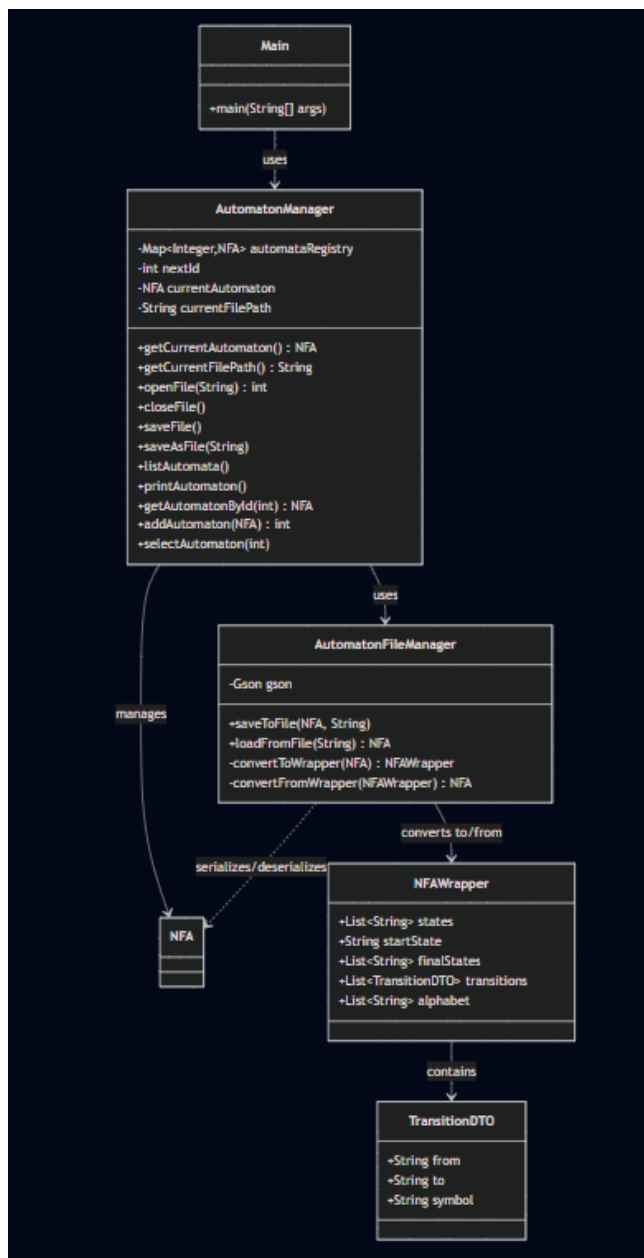
- NFA.java – основен клас на недетерминирания автомат
- State.java – представя състояние
- Transition.java – представя преход (с възможност за ε)
- NFARecognizer.java – алгоритми за разпознаване на думи
- NFADeterminism.java – проверка за детерминираност и детерминиране
- NFAOperations.java – операции union, concat, star, empty, finite

- RegexToNFA.java – преобразуване на регулярен израз в NFA

4. Пакет io – вход/изход

- AutomatonFileManager.java – четене и писане на JSON файлове
- NFAWrapper.java – JSON обвивка за сериализация с Gson





Глава 4. Реализация и тестване

4.1. Реализация на ключови класове

Клас NFA.java

```

public class NFA {
    private Set<State> states = new HashSet<>();
    private Set<Transition> transitions = new HashSet<>();
    private State startState;
    private Set<State> finalStates = new HashSet<>();
    private Set<Character> alphabet = new HashSet<>();

    public void addState(State state) {
        states.add(state);
        if (state.isFinal()) {
            finalStates.add(state);
        }
    }
}
  
```

```

}

public void addTransition(Transition transition) {
    transitions.add(transition);
    states.add(transition.getFrom());
    states.add(transition.getTo());

    if (transition.getSymbol() != null) {
        alphabet.add(transition.getSymbol());
    }

    if (transition.getTo().isFinal()) {
        finalStates.add(transition.getTo());
    }
    if (transition.getFrom().isFinal()) {
        finalStates.add(transition.getFrom());
    }
}

public void setStartState(State startState) { this.startState = startState; }
public State getStartState() { return startState; }
public Set<State> getStates() { return states; }
public Set<Transition> getTransitions() { return transitions; }
public Set<State> getFinalStates() { return finalStates; }
public Set<Character> getAlphabet() { return alphabet; }

@Override
public String toString() {
    return "NFA: " + states.size() + " states, " + transitions.size() + " transitions";
}
}

```

Това е основният клас, който представя недетерминирания краен автомат. Той съхранява цялата структура на автомата.

Член-данни:

- `Set<State> states` – множество от всички състояния
- `Set<Transition> transitions` – множество от всички преходи
- `State startState` – начално състояние
- `Set<State> finalStates` – множество от финални състояния
- `Set<Character> alphabet` – азбука на автомата

Ключови методи:

- `addState(State state)` – добавя ново състояние; ако то е финално, се добавя и в `finalStates`
- `addTransition(Transition transition)` – добавя нов преход; автоматично добавя началното и крайното състояние в `states` и обновява `alphabet` ако преходът не е `ε`
- `setStartState()` / `getStartState()` – сетър и гетър за началното състояние

- `getStates()`, `getTransitions()`, `getFinalStates()`, `getAlphabet()` – гетъри за член-данните

Клас State.java

```
public class State {
    private String name;
    private boolean isFinal;

    public State(String name, boolean isFinal) {
        this.name = name;
        this.isFinal = isFinal;
    }

    public String getName() {
        return name;
    }

    public boolean isFinal() {
        return isFinal;
    }

    public void setFinal(boolean isFinal) {
        this.isFinal = isFinal;
    }

    @Override
    public String toString() {
        return name + (isFinal ? "[FINAL]" : "");
    }
}
```

Представя едно състояние на автомата.

Член-данни:

- `String name` – уникално име на състоянието (напр. "q0", "q1")
- `boolean isFinal` – флаг, указващ дали състоянието е финално

Ключови методи:

- `getName()` – връща името на състоянието
- `isFinal()` – проверява дали състоянието е финално
- `setFinal(boolean isFinal)` – променя финалния статус
- `equals()` и `hashCode()` – необходими за работата на `HashSet` и `HashMap`

Клас Transition.java

```
public class Transition {
    private State from;
    private State to;
```

```

private Character symbol;

public Transition(State from, State to, Character symbol) {
    this.from = from;
    this.to = to;
    this.symbol = symbol;
}

public State getFrom() {
    return from;
}

public State getTo() {
    return to;
}

public Character getSymbol() {
    return symbol;
}

public boolean isEpsilon() {
    return symbol == null;
}

@Override
public String toString() {
    if (isEpsilon()) {
        return from.getName() + ": ε -> " + to.getName();
    }
    return from.getName() + ": " + symbol + " -> " + to.getName();
}
}

```

Представя преход между две състояния.

Член-данни:

- State from – начално състояние
- State to – крайно състояние
- Character symbol – символ на прехода; ако е null, това е ε-преход

Ключови методи:

- isEpsilon() – връща true, ако преходът е ε (т.е. symbol == null)
- getFrom(), getTo(), getSymbol() – гетъри
- toString() – връща текстово представяне, напр. "q0 -a-> q1" или "q0 -ε-> q1"

Клас NFARecognizer.java

```

public class NFARecognizer {

    public static boolean recognize(NFA nfa, String word) {

```

```

Set<State> currentStates = epsilonClosure(nfa, Set.of(nfa.getStartState()));

for (char c : word.toCharArray()) {
    currentStates = move(nfa, currentStates, c);
    currentStates = epsilonClosure(nfa, currentStates);
    if (currentStates.isEmpty()) {
        return false;
    }
}

for (State s : currentStates) {
    if (s.isFinal()) {
        return true;
    }
}
return false;
}

public static Set<State> epsilonClosure(NFA nfa, Set<State> states) {
    Set<State> closure = new HashSet<>(states);
    Stack<State> stack = new Stack<>();
    stack.addAll(states);

    while (!stack.isEmpty()) {
        State current = stack.pop();
        for (Transition t : nfa.getTransitions()) {
            if (t.getFrom().equals(current) && t.isEpsilon()) {
                if (!closure.contains(t.getTo())) {
                    closure.add(t.getTo());
                    stack.push(t.getTo());
                }
            }
        }
    }
    return closure;
}

public static Set<State> move(NFA nfa, Set<State> states, char c) {
    Set<State> result = new HashSet<>();
    for (State s : states) {
        for (Transition t : nfa.getTransitions()) {
            if (t.getFrom().equals(s) && !t.isEpsilon() && t.getSymbol() == c) {
                result.add(t.getTo());
            }
        }
    }
    return result;
}
}

```

Този клас съдържа алгоритмите за разпознаване на думи в NFA.

Метод `epsilonClosure(NFA nfa, Set<State> states)`

- Намира всички състояния, достижими от дадено множество състояния само чрез ϵ -преходи.
- Използва стек за обхождане в дълбочина (DFS).

Метод `move(NFA nfa, Set<State> states, char c)`

- За дадено множество от състояния и символ `c`, връща всички състояния, достижими чрез един преход със символа `c`.

Метод `recognize(NFA nfa, String word)`

- Основният алгоритъм за разпознаване:
- Изчислява ϵ -closure на началното състояние
- За всеки символ от думата:
- Изчислява `move()` със символа
- Изчислява ϵ -closure на резултата
- След обработка на всички символи, ако някое от достигнатите състояния е финално – думата се приема.

Клас `NFADeterminism.java`

```
public class NFADeterminism {

    public static boolean isDeterministic(NFA nfa) {
        for (Transition t : nfa.getTransitions()) {
            if (t.isEpsilon()) {
                return false;
            }
        }

        for (State s : nfa.getStates()) {
            Set<Character> seenSymbols = new HashSet<>();
            for (Transition t : nfa.getTransitions()) {
                if (t.getFrom().equals(s)) {
                    char symbol = t.getSymbol();
                    if (seenSymbols.contains(symbol)) {
                        return false;
                    }
                    seenSymbols.add(symbol);
                }
            }
        }
        return true;
    }

    public static NFA determinize(NFA nfa) {
        NFA dfa = new NFA();
        Map<Set<State>, State> dfaStates = new HashMap<>();

        Set<State> startSet = NFARecognizer.epsilonClosure(nfa, Set.of(nfa.getStartState()));
        State dfaStart = new State("D{" + stateSetToString(startSet) + "}",
containsFinal(startSet));
        dfa.addState(dfaStart);
        dfa.setStartState(dfaStart);
        dfaStates.put(startSet, dfaStart);
    }
}
```

```

Queue<Set<State>> queue = new LinkedList<>();
queue.add(startSet);

while (!queue.isEmpty()) {
    Set<State> currentSet = queue.poll();
    State currentState = dfaStates.get(currentSet);

    for (char c : nfa.getAlphabet()) {
        Set<State> nextSet = NFARecognizer.epsilonClosure(nfa, NFARecognizer.move(nfa,
currentSet, c));

        if (nextSet.isEmpty()) continue;

        if (!dfaStates.containsKey(nextSet)) {
            State newState = new State("D{" + stateSetToString(nextSet) + "}",
containsFinal(nextSet));
            dfa.addState(newState);
            dfaStates.put(nextSet, newState);
            queue.add(nextSet);
        }

        State nextState = dfaStates.get(nextSet);
        dfa.addTransition(new Transition(currentState, nextState, c));
    }
}
return dfa;
}

private static boolean containsFinal(Set<State> states) {
    for (State s : states) {
        if (s.isFinal()) return true;
    }
    return false;
}

private static String stateSetToString(Set<State> states) {
    List<String> names = new ArrayList<>();
    for (State s : states) {
        names.add(s.getName());
    }
    Collections.sort(names);
    return String.join(", ", names);
}
}

```

Този клас имплементира проверката за детерминираност и детерминирането на NFA до DFA.

Метод isDeterministic(NFA nfa)

Проверява две условия:

- Няма ϵ -преходи
- От всяко състояние няма два прехода с един и същ символ

Метод determinize(NFA nfa)

Имплементира subset construction алгоритъм:

- Всяко състояние в DFA представлява множество от състояния на NFA
- Започва с ϵ -closure на началното състояние на NFA
- За всеки символ от азбуката се намира следващото множество
- Процесът продължава, докато няма нови множества
- DFA състояние е финално, ако съдържа поне едно финално NFA състояние

Клас NFAOperations.java

```
public class NFAOperations {

    public static NFA union(NFA first, NFA second) {
        NFA result = new NFA();

        Map<State, State> stateMap1 = copyFromWithPrefix(first, result, "A_");
        Map<State, State> stateMap2 = copyFromWithPrefix(second, result, "B_");

        State newStart = new State("union_start", false);
        result.addState(newStart);
        result.setStartState(newStart);

        State firstStart = stateMap1.get(first.getStartState());
        State secondStart = stateMap2.get(second.getStartState());
        result.addTransition(new Transition(newStart, firstStart, null));
        result.addTransition(new Transition(newStart, secondStart, null));

        return result;
    }

    public static NFA concat(NFA first, NFA second) {
        NFA result = new NFA();

        Map<State, State> stateMap1 = copyFromWithPrefix(first, result, "A_");
        Map<State, State> stateMap2 = copyFromWithPrefix(second, result, "B_");

        State firstStart = stateMap1.get(first.getStartState());
        result.setStartState(firstStart);

        State secondStart = stateMap2.get(second.getStartState());

        for (State s : first.getFinalStates()) {
            State mappedFinal = stateMap1.get(s);
            mappedFinal.setFinal(false);
            result.addTransition(new Transition(mappedFinal, secondStart, null));
        }
    }
}
```

```

        for (State s : second.getFinalStates()) {
            State mappedFinal = stateMap2.get(s);
            mappedFinal.setFinal(true);
        }

        return result;
    }

    public static NFA star(NFA nfa) {
        NFA result = new NFA();

        Map<State, State> stateMap = copyFromWithPrefix(nfa, result, "");

        State newStart = new State("star_start", true);
        result.addState(newStart);
        result.setStartState(newStart);

        State originalStart = stateMap.get(nfa.getStartState());
        result.addTransition(new Transition(newStart, originalStart, null));

        for (State s : nfa.getFinalStates()) {
            State mappedFinal = stateMap.get(s);
            result.addTransition(new Transition(mappedFinal, originalStart, null));
        }

        return result;
    }

    public static boolean isEmptyLanguage(NFA nfa) {
        Set<State> reachable = NFARecognizer.epsilonClosure(nfa, Set.of(nfa.getStartState()));

        boolean changed;
        do {
            changed = false;
            Set<State> newReachable = new HashSet<>(reachable);
            for (State s : reachable) {
                for (char c : nfa.getAlphabet()) {
                    Set<State> next = NFARecognizer.move(nfa, Set.of(s), c);
                    for (State ns : next) {
                        if (!reachable.contains(ns)) {
                            newReachable.add(ns);
                            changed = true;
                        }
                    }
                }
            }
            reachable = newReachable;
        } while (changed);

        for (State s : reachable) {
            if (s.isFinal()) return false;
        }
        return true;
    }

    private static Map<State, State> copyFromWithPrefix(NFA source, NFA destination, String prefix)
    {
        Map<State, State> stateMap = new HashMap<>();
    }

```

```

    for (State s : source.getStates()) {
        String newName = prefix + s.getName();
        State newState = new State(newName, s.isFinal());
        destination.addState(newState);
        stateMap.put(s, newState);
    }

    for (Transition t : source.getTransitions()) {
        State newFrom = stateMap.get(t.getFrom());
        State newTo = stateMap.get(t.getTo());
        destination.addTransition(new Transition(newFrom, newTo, t.getSymbol()));
    }

    return stateMap;
}
}

```

Този клас съдържа операциите за комбиниране на автомати.

Метод union(NFA first, NFA second)

- Създава нов автомат, който приема думите, приемани от първия ИЛИ от втория автомат.
- Използва deep copy с префикси (A_, B_), за да не се променят оригиналните автомати.
- Добавя ново начално състояние с ϵ -преходи към двата стари начални възела.

Метод concat(NFA first, NFA second)

- Създава нов автомат, който приема думите xu , където x се приема от първия автомат, а y – от втория.
- Добавя ϵ -преходи от всички финални състояния на първия към началното състояние на втория.

Метод star(NFA nfa)

- Създава автомат, който приема нула или повече повторения на думите от оригиналния автомат (Kleene star).
- Добавя ново начално състояние (което е и финално) с ϵ -преход към старото начало, както и ϵ -преходи от старите финални състояния обратно към старото начало.

Метод isEmptyLanguage(NFA nfa)

- Проверява дали езикът на автомата е празен.
- Намира всички достижими състояния чрез BFS и проверява дали някое финално състояние е достижимо.

```
public class RegexToNFA {

    public static NFA fromRegex(String regex) {
        RegexParser parser = new RegexParser(regex);
        return parser.parseExpression();
    }

    private static class RegexParser {
        private final String input;
        private int pos;

        RegexParser(String input) {
            this.input = input;
            this.pos = 0;
        }

        private char peek() { return pos < input.length() ? input.charAt(pos) : '\0'; }
        private char consume() { return input.charAt(pos++); }

        private boolean matches(char c) {
            if (peek() == c) {
                pos++;
                return true;
            }
            return false;
        }

        NFA parseExpression() {
            NFA left = parseTerm();
            while (matches('|')) {
                NFA right = parseTerm();
                left = NFAOperations.union(left, right);
            }
            return left;
        }

        NFA parseTerm() {
            NFA result = parseFactor();
            while (peek() != '\0' && peek() != '|' && peek() != ')') {
                NFA next = parseFactor();
                result = NFAOperations.concat(result, next);
            }
            return result;
        }

        NFA parseFactor() {
            NFA atom = parseAtom();
            if (matches('*')) {
                return NFAOperations.star(atom);
            }
            return atom;
        }

        NFA parseAtom() {
            if (matches('(')) {
                NFA expr = parseExpression();
                if (!matches('')) {
                    return NFAOperations.concat(expr, NFAOperations.star(NFAOperations.empty()));
                }
            }
            return NFAOperations.empty();
        }
    }
}
```

```

        throw new RuntimeException("Missing closing parenthesis");
    }
    return expr;
}

char c = consume();
if (c == '\0') {
    throw new RuntimeException("Unexpected end of regex");
}

NFA nfa = new NFA();
State start = new State("s0", false);
State end = new State("s1", true);
nfa.addState(start);
nfa.addState(end);
nfa.setStartState(start);
nfa.addTransition(new Transition(start, end, c));
return nfa;
}
}
}

```

Имплементира Thompson construction – алгоритъм за преобразуване на регулярен израз в NFA.

Метод fromRegex(String regex)

- Вход: регулярен израз (напр. "a|b*", "(ab)*")
- Изход: NFA, който разпознава точно думите, описани от израза

Как работи:

- Всеки символ а се преобразува в NFA с две състояния и един преход
- ab (конкатенация) – свързва се финалното състояние на първия с началното на втория чрез ε-преход
- a|b (обединение) – ново начално състояние с ε-преходи към двата автомата
- a* (звезда) – нови ε-преходи за образуване на цикъл

Клас AutomatonManager.java

```

public class AutomatonManager {
    private Map<Integer, NFA> automataRegistry = new HashMap<>();
    private int nextId = 1;
    private NFA currentAutomaton = null;
    private String currentFilePath = null;

    public NFA getCurrentAutomaton() { return currentAutomaton; }
    public String getCurrentFilePath() { return currentFilePath; }

    public int openFile(String filePath) {
        try {
            NFA nfa = AutomatonFileManager.LoadFromFile(filePath);
            int id = nextId++;

```

```

        automataRegistry.put(id, nfa);
        currentAutomaton = nfa;
        currentFilePath = filePath;
        System.out.println("Successfully opened " + filePath + " (ID: " + id + ")");
        return id;
    } catch (Exception e) {
        System.out.println("Файлът не съществува. Създава се нов празен автомат.");
        NFA newNFA = new NFA();
        int id = nextId++;
        automataRegistry.put(id, newNFA);
        currentAutomaton = newNFA;
        currentFilePath = filePath;
        System.out.println("Successfully created new automaton " + filePath + " (ID: " + id +
    "));
        return id;
    }
}

public void closeFile() {
    if (currentAutomaton == null) {
        System.out.println("Няма отворен файл за затваряне.");
        return;
    }
    System.out.println("Successfully closed " + currentFilePath);
    currentAutomaton = null;
    currentFilePath = null;
}

public void saveFile() {
    if (currentAutomaton == null) {
        System.out.println("Няма отворен автомат за записване.");
        return;
    }
    if (currentFilePath == null) {
        System.out.println("Няма файл. Използвайте 'saveas <file>'.");
        return;
    }
    try {
        AutomatonFileManager.saveToFile(currentAutomaton, currentFilePath);
        System.out.println("Successfully saved " + currentFilePath);
    } catch (Exception e) {
        System.out.println("Грешка при запис: " + e.getMessage());
    }
}

public void saveAsFile(String newFilePath) {
    if (currentAutomaton == null) {
        System.out.println("Няма отворен автомат за записване.");
        return;
    }
    try {
        AutomatonFileManager.saveToFile(currentAutomaton, newFilePath);
        currentFilePath = newFilePath;
        System.out.println("Successfully saved " + newFilePath);
    } catch (Exception e) {
        System.out.println("Грешка при запис: " + e.getMessage());
    }
}

public void listAutomata() {
    if (automataRegistry.isEmpty()) {
        System.out.println("Няма заредени автомати.");
    }
}

```

```

        return;
    }
    System.out.println("\nЗаредени автомати:");
    for (Map.Entry<Integer, NFA> entry : automataRegistry.entrySet()) {
        int id = entry.getKey();
        NFA nfa = entry.getValue();
        boolean isCurrent = (currentAutomaton == nfa);
        String marker = isCurrent ? " <- текущ" : "";
        System.out.println("  ID " + id + ": " + nfa + marker);
    }
    System.out.println();
}

public void printAutomaton() {
    if (currentAutomaton == null) {
        System.out.println("Няма отворен автомат.");
        return;
    }
    System.out.println("\nАвтомат (ID: " + getIdOfCurrentAutomaton() + "):");
    System.out.println("  Стартово състояние: " + currentAutomaton.getStartState());
    System.out.println("  Финални състояния: " + currentAutomaton.getFinalStates());
    System.out.println("  Азбука: " + currentAutomaton.getAlphabet());
    System.out.println("\n  Преходи:");
    if (currentAutomaton.getTransitions().isEmpty()) {
        System.out.println("    (няма преходи)");
    } else {
        for (var t : currentAutomaton.getTransitions()) {
            System.out.println("    " + t);
        }
    }
    System.out.println();
}

public NFA getAutomatonById(int id) {
    return automataRegistry.get(id);
}

public int addAutomaton(NFA nfa) {
    int id = nextId++;
    automataRegistry.put(id, nfa);
    return id;
}

private int getIdOfCurrentAutomaton() {
    for (Map.Entry<Integer, NFA> entry : automataRegistry.entrySet()) {
        if (entry.getValue() == currentAutomaton) {
            return entry.getKey();
        }
    }
    return -1;
}

public void selectAutomaton(int id) {
    NFA selected = automataRegistry.get(id);
    if (selected == null) {
        System.out.println("No automaton with ID " + id);
        return;
    }

    currentAutomaton = selected;
    currentFilePath = null;
}

```

```

        System.out.println("Switched to automaton ID: " + id);
    }
}

```

Този клас управлява всички заредени автомати и текущия файл.

Член-данни:

- Map<Integer, NFA> automataRegistry – регистър на автоматите с уникални ID-та
- int nextId – следващо свободно ID
- NFA currentAutomaton – текущо отвореният автомат
- String currentFilePath – път до текущия файл

Ключови методи:

- openFile(String filePath) – зарежда автомат от файл (или създава нов)
- closeFile() – затваря текущия автомат
- saveFile() – записва в текущия файл
- saveAsFile(String filePath) – записва в нов файл
- listAutomata() – извежда списък с всички ID-та
- printAutomaton() – извежда всички преходи
- selectAutomaton(int id) – сменя текущия автомат

Клас AutomatonFileManager.java

```

public class AutomatonFileManager {

    private static final Gson gson = new GsonBuilder().setPrettyPrinting().create();

    public static void saveToFile(NFA nfa, String filePath) throws IOException {
        NFAWrapper wrapper = convertToWrapper(nfa);
        try (Writer writer = new FileWriter(filePath)) {
            gson.toJson(wrapper, writer);
        }
    }

    public static NFA loadFromFile(String filePath) throws IOException {
        try (Reader reader = new FileReader(filePath)) {
            NFAWrapper wrapper = gson.fromJson(reader, NFAWrapper.class);
            return convertFromWrapper(wrapper);
        }
    }

    private static NFAWrapper convertToWrapper(NFA nfa) {
        NFAWrapper wrapper = new NFAWrapper();
    }
}

```



```

wrapper.states = new ArrayList<>();
wrapper.finalStates = new ArrayList<>();

wrapper.alphabet = new ArrayList<>();
for (Character c : nfa.getAlphabet()) {
    wrapper.alphabet.add(String.valueOf(c));
}

Map<State, String> stateNames = new HashMap<>();
for (State s : nfa.getStates()) {
    stateNames.put(s, s.getName());
    wrapper.states.add(s.getName());
    if (s.isFinal()) {
        wrapper.finalStates.add(s.getName());
    }
}
wrapper.startState = nfa.getStartState().getName();

wrapper.transitions = new ArrayList<>();
for (Transition t : nfa.getTransitions()) {
    NFAWrapper.TransitionInfo info = new NFAWrapper.TransitionInfo();
    info.from = t.getFrom().getName();
    info.to = t.getTo().getName();
    info.symbol = t.isEpsilon() ? null : String.valueOf(t.getSymbol());
    wrapper.transitions.add(info);
}
return wrapper;
}

private static NFA convertFromWrapper(NFAWrapper wrapper) {
    NFA nfa = new NFA();
    Map<String, State> stateMap = new HashMap<>();

    for (String stateName : wrapper.states) {
        boolean isFinal = wrapper.finalStates != null &&
wrapper.finalStates.contains(stateName);
        State state = new State(stateName, isFinal);
        stateMap.put(stateName, state);
        nfa.addState(state);
        if (wrapper.startState.equals(stateName)) {
            nfa.setStartState(state);
        }
    }

    if (wrapper.transitions != null) {
        for (NFAWrapper.TransitionInfo info : wrapper.transitions) {
            State from = stateMap.get(info.from);
            State to = stateMap.get(info.to);
            Character symbol = (info.symbol == null || info.symbol.equals("null")) ||
info.symbol.isEmpty()
                ? null
                : info.symbol.charAt(0);
            nfa.addTransition(new Transition(from, to, symbol));
        }
    }
}

```

```

    }
}

return nfa;
}
}

```

Този клас отговаря за сериализацията и десериализацията на автоматите във JSON формат. Използва библиотеката Gson.

Член-данни:

- Gson gson – инстанция на Gson с форматиране за четим изход

Ключови методи:

- saveToFile(NFA nfa, String filePath) – записва автомат във файл
- loadFromFile(String filePath) – зарежда автомат от файл
- convertToWrapper(NFA nfa) – преобразува NFA в NFAWrapper (за JSON)
- convertFromWrapper(NFAWrapper wrapper) – преобразува NFAWrapper обратно в NFA

Клас NFAWrapper.java

```

public class NFAWrapper {
    public List<String> states;
    public String startState;
    public List<String> finalStates;
    public List<TransitionInfo> transitions;
    public List<String> alphabet;

    public static class TransitionInfo {
        public String from;
        public String to;
        public String symbol;
    }
}

```

Това е помощен клас за Gson. Тъй като Gson не може директно да сериализира обектите State и Transition (заради кръгови референции), използваме NFAWrapper като "чисто" JSON представяне на автомата.

Член-данни:

- List<String> states – списък с имената на всички състояния
- String startState – име на началното състояние
- List<String> finalStates – списък с имената на финалните състояния

- List<TransitionInfo> transitions – списък с преходите
- List<String> alphabet – списък с азбуката

Вътрешен клас TransitionInfo:

- String from – начално състояние на прехода
- String to – крайно състояние на прехода
- String symbol – символ на прехода (null за ε)

Клас CLI.java

```
public class CLI {
    private final AutomatonManager manager = new AutomatonManager();
    private final CommandFactory factory = new CommandFactory(manager);

    public void start() {
        System.out.println("=== NFA Command Line Interface ===");
        System.out.println("Type 'HELP' for available commands\n");

        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.print("> ");
            String input = scanner.nextLine();

            input = input == null ? "" : input.trim();
            if (input.isEmpty()) {
                continue;
            }

            CommandParser.Command parsedCmd = CommandParser.parse(input);

            if (parsedCmd.getName().isEmpty()) {
                continue;
            }

            Command command = factory.createCommand(parsedCmd.getName(), parsedCmd.getArgs());

            if (command != null) {
                try {
                    command.execute();
                } catch (Exception e) {
                    System.out.println("Unexpected error: " + e.getMessage());
                    System.out.println("Please check your input and try again.");
                }
            }
        }
    }
}
```

Главният цикъл на програмата. Чете потребителски вход, използва CommandParser за парсване, CommandFactory за създаване на команда и я изпълнява.

Член-данни:

- AutomatonManager manager – мениджър за автоматите
- CommandFactory factory – фабрика за команди

Ключови методи:

- start() – основният метод, който стартира CLI цикъла

Клас Command.java

```
public interface Command {  
    void execute();  
}
```

Интерфейс, който дефинира метода execute(). Всички конкретни команди (OpenCommand, CloseCommand, UnionCommand и т.н.) го имплементират. Това позволява командите да се третират еднородно и улеснява добавянето на нови команди.

Клас CommandFactory.java

```
public class CommandFactory {  
    private final AutomatonManager manager;  
    private final Map<String, Function<List<String>, Command>> commandCreators;  
  
    public CommandFactory(AutomatonManager manager) {  
        this.manager = manager;  
        this.commandCreators = new HashMap<>();  
        commandCreators.put("help", args -> new HelpCommand());  
        commandCreators.put("exit", args -> new ExitCommand());  
        commandCreators.put("close", args -> new CloseCommand(manager));  
        commandCreators.put("list", args -> new ListCommand(manager));  
        commandCreators.put("print", args -> new PrintCommand(manager));  
        commandCreators.put("save", args -> new SaveCommand(manager));  
        commandCreators.put("star", args -> new StarCommand(manager));  
        commandCreators.put("deterministic", args -> new DeterministicCommand(manager));  
        commandCreators.put("determinize", args -> new DeterminizeCommand(manager));  
        commandCreators.put("empty", args -> new EmptyCommand(manager));  
  
        commandCreators.put("open", args -> {  
            if (args.isEmpty()) {  
                System.out.println("Грешка: open <file> - липсва път към файл");  
                return null;  
            }  
            if (args.size() > 1) {  
                System.out.println("Грешка: open приема само 1 аргумент, а Вие подадохте " +  
args.size());  
                return null;  
            }  
            return new OpenCommand(manager, args.get(0));  
        });  
    }  
}
```

```

        commandCreators.put("saveas", args -> {
            if (args.isEmpty()) {
                System.out.println("Грешка: saveas <file> - липсва път към файл");
                return null;
            }
            if (args.size() > 1) {
                System.out.println("Грешка: saveas приема само 1 аргумент, а Вие подадохте " +
args.size());
                return null;
            }
            return new SaveAsCommand(manager, args.get(0));
        });

        commandCreators.put("recognize", args -> {
            if (args.isEmpty()) {
                System.out.println("Грешка: recognize <word> - липсва дума");
                return null;
            }
            if (args.size() > 1) {
                System.out.println("Грешка: recognize приема само 1 аргумент, а Вие подадохте " +
args.size());
                return null;
            }
            return new RecognizeCommand(manager, args.get(0));
        });

        commandCreators.put("reg", args -> {
            if (args.isEmpty()) {
                System.out.println("Грешка: reg <regex> - липсва регулярен израз");
                return null;
            }
            if (args.size() > 1) {
                System.out.println("Грешка: reg приема само 1 аргумент, а Вие подадохте " +
args.size());
                return null;
            }
            return new RegexCommand(manager, args.get(0));
        });

        commandCreators.put("select", args -> {
            if (args.isEmpty()) {
                System.out.println("Грешка: select <id> - липсва ID");
                return null;
            }
            if (args.size() > 1) {
                System.out.println("Грешка: select приема само 1 аргумент, а Вие подадохте " +
args.size());
                return null;
            }
            try {
                int id = Integer.parseInt(args.get(0));
                if (manager.getAutomatonById(id) == null) {
                    System.out.println("Грешка: Няма автомат с ID " + id);
                    return null;
                }
                return new SelectCommand(manager, id);
            } catch (NumberFormatException e) {
                System.out.println("Грешка: ID трябва да е число");
                return null;
            }
        });
    });

```

```

        commandCreators.put("union", args -> {
            if (args.isEmpty()) {
                System.out.println("Грешка: union <id> - липсва ID");
                return null;
            }
            if (args.size() > 1) {
                System.out.println("Грешка: union приема само 1 аргумент, а Вие подадохте " +
args.size());
                return null;
            }
            try {
                int id = Integer.parseInt(args.get(0));
                return new UnionCommand(manager, id);
            } catch (NumberFormatException e) {
                System.out.println("Грешка: ID трябва да е число");
                return null;
            }
        });

        commandCreators.put("concat", args -> {
            if (args.isEmpty()) {
                System.out.println("Грешка: concat <id> - липсва ID");
                return null;
            }
            if (args.size() > 1) {
                System.out.println("Грешка: concat приема само 1 аргумент, а Вие подадохте " +
args.size());
                return null;
            }
            try {
                int id = Integer.parseInt(args.get(0));
                return new ConcatCommand(manager, id);
            } catch (NumberFormatException e) {
                System.out.println("Грешка: ID трябва да е число");
                return null;
            }
        });
    }

    public Command createCommand(String name, List<String> args) {
        Function<List<String>, Command> creator = commandCreators.get(name);
        if (creator == null) {
            System.out.println("Неизвестна команда: " + name);
            return null;
        }
        return creator.apply(args);
    }
}

```

Фабрика, която по име на команда и списък от аргументи връща съответния команден обект. Използва модерния синтаксис return switch с yield за по-чист и безопасен код.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

Ключови методи:

- `createCommand(String name, List<String> args)` – връща команден обект или `null` при грешка

Клас `CommandHandler.java`

```
public class CommandHandler {
    private final AutomatonManager manager;
    private final Map<String, CommandExecutor> executors;

    private interface CommandExecutor {
        void execute(Command cmd);
    }

    public CommandHandler(AutomatonManager manager) {
        this.manager = manager;
        this.executors = new HashMap<>();

        executors.put("help", cmd -> HelpPrinter.print());
        executors.put("exit", cmd -> { System.out.println("Излизане от програмата...");
        System.exit(0); });
        executors.put("close", cmd -> manager.closeFile());
        executors.put("list", cmd -> manager.listAutomata());
        executors.put("print", cmd -> manager.printAutomaton());
        executors.put("save", cmd -> manager.saveFile());
        executors.put("deterministic", cmd -> handleDeterministic());
        executors.put("empty", cmd -> handleEmpty());
        executors.put("star", cmd -> handleStar());
        executors.put("determinize", cmd -> handleDeterminize());

        executors.put("open", cmd -> {
            if (cmd.getArgCount() < 1) {
                System.out.println("Грешка: open <file>");
                return;
            }
            manager.openFile(cmd.getArg(0));
        });
        executors.put("saveas", cmd -> {
            if (cmd.getArgCount() < 1) {
                System.out.println("Грешка: saveas <file>");
                return;
            }
            manager.saveAsFile(cmd.getArg(0));
        });
        executors.put("recognize", cmd -> handleRecognize(cmd));
        executors.put("union", cmd -> handleUnion(cmd));
        executors.put("concat", cmd -> handleConcat(cmd));
        executors.put("reg", cmd -> handleRegex(cmd));
    }

    public void execute(Command cmd) {
        CommandExecutor executor = executors.get(cmd.getName());
        if (executor == null) {
            System.out.println("Неизвестна команда: " + cmd.getName());
            return;
        }
    }
}
```

```

        executor.execute(cmd);
    }

    private void handleRecognize(Command cmd) {
        NFA nfa = manager.getCurrentAutomaton();
        if (nfa == null) {
            System.out.println("Няма отворен автомат.");
            return;
        }
        if (cmd.getArgCount() < 1) {
            System.out.println("Грешка: recognize <word>");
            return;
        }
        String word = cmd.getArg(0);
        boolean accepted = NFARecognizer.recognize(nfa, word);
        System.out.println("Думата \"" + word + "\" " + (accepted ? "СЕ приема" : "НЕ СЕ приема"));
    }

    private void handleDeterministic() {
        NFA nfa = manager.getCurrentAutomaton();
        if (nfa == null) {
            System.out.println("Няма отворен автомат.");
            return;
        }
        System.out.println("Автоматът е " + (NFADeterminism.isDeterministic(nfa) ? "ДЕТЕРМИНИРАН" :
"НЕДЕТЕРМИНИРАН"));
    }

    private void handleEmpty() {
        NFA nfa = manager.getCurrentAutomaton();
        if (nfa == null) {
            System.out.println("Няма отворен автомат.");
            return;
        }
        System.out.println("Езикът на автомата е " + (NFAOperations.isEmptyLanguage(nfa) ?
"ПРАЗЕН" : "НЕ Е ПРАЗЕН"));
    }

    private void handleUnion(Command cmd) {
        NFA current = manager.getCurrentAutomaton();
        if (current == null) {
            System.out.println("Няма отворен автомат.");
            return;
        }
        if (cmd.getArgCount() < 1) {
            System.out.println("Грешка: union <id>");
            return;
        }
        try {
            int otherId = Integer.parseInt(cmd.getArg(0));
            NFA other = manager.getAutomatonById(otherId);
            if (other == null) {
                System.out.println("Няма автомат с ID " + otherId);
                return;
            }
            NFA result = NFAOperations.union(current, other);
            int newId = manager.addAutomaton(result);
            System.out.println("Обединението е създадено с ID: " + newId);
        } catch (NumberFormatException e) {
            System.out.println("Грешка: ID трябва да е число");
        }
    }

```



```

}

private void handleConcat(Command cmd) {
    NFA current = manager.getCurrentAutomaton();
    if (current == null) {
        System.out.println("Няма отворен автомат.");
        return;
    }
    if (cmd.getArgCount() < 1) {
        System.out.println("Грешка: concat <id>");
        return;
    }
    try {
        int otherId = Integer.parseInt(cmd.getArg(0));
        NFA other = manager.getAutomatonById(otherId);
        if (other == null) {
            System.out.println("Няма автомат с ID " + otherId);
            return;
        }
        NFA result = NFAOperations.concat(current, other);
        int newId = manager.addAutomaton(result);
        System.out.println("Конкатенацията е създадена с ID: " + newId);
    } catch (NumberFormatException e) {
        System.out.println("Грешка: ID трябва да е число");
    }
}

private void handleStar() {
    NFA current = manager.getCurrentAutomaton();
    if (current == null) {
        System.out.println("Няма отворен автомат.");
        return;
    }
    NFA result = NFAOperations.star(current);
    int newId = manager.addAutomaton(result);
    System.out.println("Звездата на автомата е създадена с ID: " + newId);
}

private void handleDeterminize() {
    NFA current = manager.getCurrentAutomaton();
    if (current == null) {
        System.out.println("Няма отворен автомат.");
        return;
    }
    NFA dfa = NFADeterminism.determinize(current);
    int newId = manager.addAutomaton(dfa);
    System.out.println("Детерминираният автомат е създаден с ID: " + newId);
}

private void handleRegex(Command cmd) {
    if (cmd.getArgCount() < 1) {
        System.out.println("Грешка: reg <regex>");
        return;
    }
    try {
        NFA nfa = RegexToNFA.fromRegex(cmd.getArg(0));
        int newId = manager.addAutomaton(nfa);
        System.out.println("Автоматът от регулярен израз е създаден с ID: " + newId);
    } catch (Exception e) {
        System.out.println("Грешка при парсване на regex: " + e.getMessage());
    }
}

```

```
}  
}
```

Обработва изпълнението на командите. Получава вече парсната команда от `CommandParser` и извиква съответния метод на `AutomatonManager`.

Член-данни:

- `AutomatonManager manager` – мениджър за автоматите

Ключови методи:

- `execute(CommandParser.Command cmd)` – изпълнява съответната команда

Клас `CommandParser.java`

```
public class CommandParser {  
  
    public static Command parse(String input) {  
        if (input == null || input.trim().isEmpty()) {  
            return new Command("", new ArrayList<>());  
        }  
  
        String[] parts = input.trim().split("\\s+");  
        String commandName = parts[0].toLowerCase();  
        List<String> args = new ArrayList<>();  
  
        for (int i = 1; i < parts.length; i++) {  
            args.add(parts[i]);  
        }  
  
        return new Command(commandName, args);  
    }  
  
    public static class Command {  
        private final String name;  
        private final List<String> args;  
  
        public Command(String name, List<String> args) {  
            this.name = name;  
            this.args = args;  
        }  
  
        public String getName() {  
            return name;  
        }  
  
        public List<String> getArgs() {  
            return args;  
        }  
    }  
}
```

```

    public int getArgCount() {
        return args.size();
    }

    public String getArg(int index) {
        if (index >= 0 && index < args.size()) {
            return args.get(index);
        }
        return null;
    }

    @Override
    public String toString() {
        return "Command{" + name + " " + args + "}";
    }
}

```

Парсва входния ред от потребителя. Разделя го на име на команда и списък от аргументи. Връща обект от тип Command (вътрешен клас), който се използва от CommandHandler.

Ключови методи:

- parse(String input) – парсва входния низ и връща обект Command

Вътрешен клас Command:

- getName() – връща името на командата
- getArgs() – връща списъка с аргументи
- getArgCount() – връща броя на аргументите
- getArg(int index) – връща конкретен аргумент по индекс

Клас CommandType.java

```

public enum CommandType {
    HELP("help", "Displays this help message"),
    EXIT("exit", "Exits the program"),
    OPEN("open", "Opens or creates an automaton from a file"),
    CLOSE("close", "Closes the currently open automaton"),
    LIST("list", "Lists all loaded automata with their IDs"),
    PRINT("print", "Prints all transitions of the current automaton"),
    SAVE("save", "Saves the current automaton to the opened file"),
    SAVEAS("saveas", "Saves the current automaton to a new file"),
    RECOGNIZE("recognize", "Checks if a word is accepted by the automaton"),
    DETERMINISTIC("deterministic", "Checks if the automaton is deterministic"),
    DETERMINIZE("determinize", "Converts NFA to DFA (subset construction)",
    EMPTY("empty", "Checks if the language of the automaton is empty"),
    UNION("union", "Creates union of current automaton with another automaton"),
    CONCAT("concat", "Creates concatenation of current automaton with another automaton"),
    STAR("star", "Creates Kleene star of the current automaton"),
    SELECT("select", "Switches current automaton to the specified ID"),
    REG("reg", "Creates an automaton from a regular expression");
}

```

```

private final String command;
private final String description;

CommandType(String command, String description) {
    this.command = command;
    this.description = description;
}

public String getCommand() {
    return command;
}

public String getDescription() {
    return description;
}

public static CommandType fromString(String text) {
    for (CommandType cmd : CommandType.values()) {
        if (cmd.command.equalsIgnoreCase(text)) {
            return cmd;
        }
    }
    return null;
}
}

```

Изброен тип, който съдържа всички поддържани команди заедно с техните описания. Използва се от HelpPrinter за генериране на помощното меню.

Член-данни:

- String command – името на командата
- String description – описание на командата

Ключови методи:

- getCommand() – връща името на командата
- getDescription() – връща описанието
- fromString(String text) – връща CommandType по име

Клас HelpPrinter.java

```

public class HelpPrinter {

    public static void print() {
        System.out.println("\nNFA COMMAND LINE INTERFACE");
        System.out.println("=====");
        System.out.println("Usage: <COMMAND> [arguments]");
        System.out.println("\nAvailable commands:\n");

        int maxLength = 0;
    }
}

```

```

    for (CommandType cmd : CommandType.values()) {
        maxLength = Math.max(maxLength, cmd.getCommand().length());
    }

    int padding = maxLength + 4;

    for (CommandType cmd : CommandType.values()) {
        String command = cmd.getCommand().toUpperCase();
        String description = cmd.getDescription();

        System.out.printf("  %-"+padding+"s %s%n", command, description);
    }

    System.out.println("\nExamples:");
    System.out.println("  > OPEN test.json");
    System.out.println("  > RECOGNIZE ab");
    System.out.println("  > UNION 1");
    System.out.println("  > REG a|b*");
    System.out.println("\nFor more details, visit the project documentation.\n");
}
}

```

Извежда помощното меню. Обхожда enum CommandType и форматира командите в две колони – лява (командата) и дясна (описанието).

Ключови методи:

- print() – извежда форматиран списък с всички команди

Клас OpenCommand.java

```

public class OpenCommand implements Command {
    private final AutomatonManager manager;
    private final String filePath;

    public OpenCommand(AutomatonManager manager, String filePath) {
        this.manager = manager;
        this.filePath = filePath;
    }

    @Override
    public void execute() {
        manager.openFile(filePath);
    }
}

```

Конкретна команда за отваряне на файл. Приема AutomatonManager и път до файл. В execute() извиква съответния метод на мениджъра.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

- String filePath – път до файла

Ключови методи:

- execute() – изпълнява командата (отваря файл)

Клас CloseCommand.java

```
public class CloseCommand implements Command {  
    private final AutomatonManager manager;  
  
    public CloseCommand(AutomatonManager manager) {  
        this.manager = manager;  
    }  
  
    @Override  
    public void execute() {  
        manager.closeFile();  
    }  
}
```

Конкретна команда за затваряне на файл. Няма аргументи.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

Ключови методи:

- execute() – изпълнява командата (затваря текущия файл)

Клас SaveCommand.java

```
public class SaveCommand implements Command {  
    private final AutomatonManager manager;  
  
    public SaveCommand(AutomatonManager manager) {  
        this.manager = manager;  
    }  
  
    @Override  
    public void execute() {  
        manager.saveFile();  
    }  
}
```

Конкретна команда за записване на промените в текущо отворения файл. Няма аргументи.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

Ключови методи:

- execute() – изпълнява командата (записва текущия автомат във файла, от който е зареден)

Как работи:

- Командата няма аргументи
- Извиква метода saveFile() на AutomatonManager
- Ако няма отворен файл или няма текущ автомат, AutomatonManager извежда подходящо съобщение за грешка

Клас SaveAsCommand.java

```
public class SaveAsCommand implements Command {
    private final AutomatonManager manager;
    private final String filePath;

    public SaveAsCommand(AutomatonManager manager, String filePath) {
        this.manager = manager;
        this.filePath = filePath;
    }

    @Override
    public void execute() {
        manager.saveAsFile(filePath);
    }
}
```

Конкретна команда за записване на текущия автомат в нов файл. Приема един аргумент – пътя до новия файл.

Член-данни:

- AutomatonManager manager – мениджър за автоматите
- String filePath – път до файла, в който ще се запише автоматът

Ключови методи:

- execute() – изпълнява командата (записва текущия автомат в посочения файл)

Как работи:

- Командата приема един аргумент – път до файл

- Извиква метода saveAsFile(filePath) на AutomatonManager
- След успешен запис, текущият файл се променя на новия (за последващи save команди)
- Ако няма отворен автомат, AutomatonManager извежда подходящо съобщение за грешка

Клас ListCommand.java

```
public class ListCommand implements Command {
    private final AutomatonManager manager;

    public ListCommand(AutomatonManager manager) {
        this.manager = manager;
    }

    @Override
    public void execute() {
        manager.listAutomata();
    }
}
```

Конкретна команда за извеждане на списък с всички заредени автомати в паметта. Няма аргументи.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

Ключови методи:

execute() – изпълнява командата (извежда списък с ID-тата на всички автомати)

Как работи:

- Командата няма аргументи
- Извиква метода listAutomata() на AutomatonManager
- Методът обхожда регистъра automataRegistry и извежда:
 - ID на всеки автомат
 - Брой състояния и преходи
 - Отметка <- текущ за текущо отворения автомат
 - Ако няма заредени автомати, се извежда съобщение "Няма заредени автомати"

Клас PrintCommand.java


```

public class PrintCommand implements Command {
    private final AutomatonManager manager;

    public PrintCommand(AutomatonManager manager) {
        this.manager = manager;
    }

    @Override
    public void execute() {
        manager.printAutomaton();
    }
}

```

Конкретна команда за извеждане на подробна информация за текущо отворения автомат. Няма аргументи.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

Ключови методи:

- execute() – изпълнява командата (извежда информация за текущия автомат)

Как работи:

- Командата няма аргументи
- Извиква метода printAutomaton() на AutomatonManager
- Ако няма отворен автомат, се извежда съобщение "Няма отворен автомат"

Клас RecognizeCommand.java

```

public class RecognizeCommand implements Command {
    private final AutomatonManager manager;
    private final String word;

    public RecognizeCommand(AutomatonManager manager, String word) {
        this.manager = manager;
        this.word = word;
    }

    @Override
    public void execute() {
        NFA nfa = manager.getCurrentAutomaton();
        if (nfa == null) {
            System.out.println("Няма отворен автомат.");
            return;
        }
        boolean accepted = NFARecognizer.recognize(nfa, word);
        System.out.println("Думата \"" + word + "\" " + (accepted ? "СЕ приема" : "НЕ СЕ приема"));
    }
}

```

```
}  
}
```

Конкретна команда за проверка дали дадена дума принадлежи на езика на текущо отворения автомат. Приема един аргумент – думата за проверка.

Член-данни:

- AutomatonManager manager – мениджър за автоматите
- String word – думата, която ще се проверява

Ключови методи:

- execute() – изпълнява командата (проверява дали думата се приема от автомата)

Как работи:

- Командата приема един аргумент – думата за проверка
- Взема текущия автомат от AutomatonManager
- Извиква метода recognize() на NFARecognizer, който:
 - Изчислява ϵ -closure на началното състояние
 - За всеки символ от думата последователно прилага move() и epsilonClosure()
 - След обработка на всички символи, проверява дали някое от достигнатите състояния е финално
 - Извежда резултата – дали думата СЕ приема или НЕ СЕ приема

Клас DeterministicCommand.java

```
public class DeterministicCommand implements Command {  
    private final AutomatonManager manager;  
  
    public DeterministicCommand(AutomatonManager manager) {  
        this.manager = manager;  
    }  
  
    @Override  
    public void execute() {  
        NFA nfa = manager.getCurrentAutomaton();  
        if (nfa == null) {  
            System.out.println("Няма отворен автомат.");  
            return;  
        }  
        System.out.println("Автоматът е " + (NFADeterminism.isDeterministic(nfa) ? "ДЕТЕРМИНИРАН" :  
"НЕДЕТЕРМИНИРАН"));  
    }  
}
```

Конкретна команда за проверка дали текущо отвореният автомат е детерминиран. Няма аргументи.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

Ключови методи:

- execute() – изпълнява командата (проверява дали автоматът е детерминиран)

Как работи:

- Командата няма аргументи
- Взема текущия автомат от AutomatonManager
- Извиква метода isDeterministic() на NFADeterminism, който проверява две

условия:

- Няма ϵ -преходи – обхожда всички преходи и проверява дали някой е ϵ -преход (symbol == null)
- От всяко състояние няма два прехода с един и същ символ – за всяко състояние проверява дали няма повторение на символи в преходите
- Извежда резултата – "ДЕТЕРМИНИРАН" или "НЕДЕТЕРМИНИРАН"

Клас DeterminizeCommand.java

```
public class DeterminizeCommand implements Command {
    private final AutomatonManager manager;

    public DeterminizeCommand(AutomatonManager manager) {
        this.manager = manager;
    }

    @Override
    public void execute() {
        NFA nfa = manager.getCurrentAutomaton();
        if (nfa == null) {
            System.out.println("Няма отворен автомат.");
            return;
        }
        NFA dfa = NFADeterminism.determinize(nfa);
        int newId = manager.addAutomaton(dfa);
        System.out.println("Детерминираният автомат е създаден с ID: " + newId);
    }
}
```

Конкретна команда за детерминиране на текущо отворения автомат. Няма аргументи.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

Ключови методи:

- execute() – изпълнява командата (детерминира автомата и създава нов DFA)

Как работи:

- Командата няма аргументи
- Взема текущия автомат от AutomatonManager
- Извиква метода determinize() на NFADeterminism, който имплементира subset

construction алгоритъм:

- Всяко състояние в DFA представлява множество от състояния на NFA
- Започва с ϵ -closure на началното състояние на NFA
- За всеки символ от азбуката се намира следващото множество
- Процесът продължава, докато няма нови множества
- DFA състояние е финално, ако съдържа поне едно финално NFA състояние
- Новосъздаденият DFA се добавя в регистъра на AutomatonManager с автоматично

генерирано ID

- Извежда ID на новия детерминиран автомат

Клас EmptyCommand.java

```
public class EmptyCommand implements Command {
    private final AutomatonManager manager;

    public EmptyCommand(AutomatonManager manager) {
        this.manager = manager;
    }

    @Override
    public void execute() {
        NFA nfa = manager.getCurrentAutomaton();
        if (nfa == null) {
            System.out.println("Няма отворен автомат.");
            return;
        }
        System.out.println("Езикът на автомата е " + (NFAOperations.isEmptyLanguage(nfa) ?
"ПРАЗЕН" : "НЕ Е ПРАЗЕН"));
    }
}
```

Конкретна команда за проверка дали езикът на текущо отворения автомат е празен. Няма аргументи.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

Ключови методи:

- execute() – изпълнява командата (проверява дали езикът на автомата е празен)

Как работи:

- Командата няма аргументи
- Взема текущия автомат от AutomatonManager
- Извиква метода isEmptyLanguage() на NFAOperations, който:
 - Намира всички достижими състояния от началното (включително чрез ε-преходи)
 - Използва итеративен процес за разширяване на достижимите състояния чрез символите от азбуката
 - Проверява дали някое от достижимите състояния е финално
 - Ако има достижимо финално състояние – езикът НЕ Е ПРАЗЕН
 - Ако няма достижимо финално състояние – езикът е ПРАЗЕН
 - Извежда резултата

Клас UnionCommand.java

```
public class UnionCommand implements Command {
    private final AutomatonManager manager;
    private final int otherId;

    public UnionCommand(AutomatonManager manager, int otherId) {
        this.manager = manager;
        this.otherId = otherId;
    }

    @Override
    public void execute() {
        NFA current = manager.getCurrentAutomaton();
        if (current == null) {
            System.out.println("Няма отворен автомат.");
            return;
        }
        NFA other = manager.getAutomatonById(otherId);
        if (other == null) {
            System.out.println("Няма автомат с ID " + otherId);
            return;
        }
        NFA result = NFAOperations.union(current, other);
        int newId = manager.addAutomaton(result);
        System.out.println("Обединението е създадено с ID: " + newId);
    }
}
```

```
}  
}
```

Конкретна команда за намиране на обединението на текущия автомат с друг автомат по ID. Приема един аргумент – ID на втория автомат.

Член-данни:

- AutomatonManager manager – мениджър за автоматите
- int otherId – идентификатор на другия автомат за обединение

Ключови методи:

- execute() – изпълнява командата (създава обединение на двата автомата)

Как работи:

- Командата приема един аргумент – ID на друг автомат
- Взема текущия автомат от AutomatonManager
- Взема другия автомат от регистъра по подаденото ID
- Извиква метода union() на NFAOperations, който:
 - Създава дълбоко копие на двата автомата с префикси (A_, B_), за да не се променят оригиналите
 - Създава ново начално състояние (union_start)
 - Добавя ε-преходи от новото начало към двата стари начални възела
- Новосъздаденият автомат (обединението) се добавя в регистъра с автоматично генерирано ID
- Извежда ID на новия автомат

Клас ConcatCommand.java

```
public class ConcatCommand implements Command {  
    private final AutomatonManager manager;  
    private final int otherId;  
  
    public ConcatCommand(AutomatonManager manager, int otherId) {  
        this.manager = manager;  
        this.otherId = otherId;  
    }  
  
    @Override  
    public void execute() {  
        NFA current = manager.getCurrentAutomaton();  
        if (current == null) {  
            System.out.println("Няма отворен автомат.");  
            return;  
        }  
    }  
}
```

```

NFA other = manager.getAutomatonById(otherId);
if (other == null) {
    System.out.println("Няма автомат с ID " + otherId);
    return;
}
NFA result = NFAOperations.concat(current, other);
int newId = manager.addAutomaton(result);
System.out.println("Конкатенацията е създадена с ID: " + newId);
}
}

```

Конкретна команда за намиране на конкатенацията на текущия автомат с друг автомат по ID. Приема един аргумент – ID на втория автомат.

Член-данни:

- AutomatonManager manager – мениджър за автоматите
- int otherId – идентификатор на другия автомат за конкатенация

Ключови методи:

- execute() – изпълнява командата (създава конкатенация на двата автомата)

Как работи:

- Командата приема един аргумент – ID на друг автомат
- Взема текущия автомат от AutomatonManager
- Взема другия автомат от регистъра по подаденото ID
- Извиква метода concat() на NFAOperations, който:
 - Създава дълбоко копие на двата автомата с префикси (A_, B_), за да не се променят оригиналите
 - Запазва началното състояние на първия автомат
 - Добавя ε-преходи от всички финални състояния на първия към началното състояние на втория
 - Финалните състояния на втория остават финални, а тези на първия вече не са финални
- Новосъздаденият автомат (конкатенацията) се добавя в регистъра с автоматично генерирано ID
- Извежда ID на новия автомат

Клас StarCommand.java

```

public class StarCommand implements Command {
    private final AutomatonManager manager;

```

```

public StarCommand(AutomatonManager manager) {
    this.manager = manager;
}

@Override
public void execute() {
    NFA current = manager.getCurrentAutomaton();
    if (current == null) {
        System.out.println("Няма отворен автомат.");
        return;
    }
    NFA result = NFAOperations.star(current);
    int newId = manager.addAutomaton(result);
    System.out.println("Звездата на автомата е създадена с ID: " + newId);
}
}

```

Конкретна команда за намиране на позитивна обвивка (Kleene star) на текущия автомат. Няма аргументи.

Член-данни:

- AutomatonManager manager – мениджър за автоматите

Ключови методи:

- execute() – изпълнява командата (създава звезда на автомата)

Как работи:

- Командата няма аргументи
- Взема текущия автомат от AutomatonManager
- Извиква метода star() на NFAOperations, който:
 - Създава дълбоко копие на автомата
 - Създава ново начално състояние (star_start), което е и финално
 - Добавя ε-преход от новото начало към старото начално състояние
 - Добавя ε-преходи от всички стари финални състояния обратно към старото

начално състояние

- Новосъздаденият автомат (звездата) се добавя в регистъра с автоматично генерирано ID
- Извежда ID на новия автомат

Клас RegexCommand.java

```

public class RegexCommand implements Command {
    private final AutomatonManager manager;
    private final String regex;
}

```



```

public RegexCommand(AutomatonManager manager, String regex) {
    this.manager = manager;
    this.regex = regex;
}

@Override
public void execute() {
    try {
        NFA nfa = RegexToNFA.fromRegex(regex);
        int newId = manager.addAutomaton(nfa);
        System.out.println("Автоматът от регулярен израз е създаден с ID: " + newId);
    } catch (Exception e) {
        System.out.println("Грешка при парсване на regex: " + e.getMessage());
    }
}
}

```

Конкретна команда за създаване на автомат от регулярен израз (теорема на Клини). Приема един аргумент – регулярния израз.

Член-данни:

- AutomatonManager manager – мениджър за автоматите
- String regex – регулярният израз, който ще се преобразува в автомат

Ключови методи:

- execute() – изпълнява командата (създава NFA от регулярен израз)

Как работи:

- Командата приема един аргумент – регулярен израз (напр. "a|b*", "(ab)*")
- Извиква метода fromRegex() на RegexToNFA, който имплементира Thompson

construction:

- Всеки символ а се преобразува в NFA с две състояния и един преход
- ab (конкатенация) – свързва се финалното състояние на първия с началното на втория чрез ε-преход
- a|b (обединение) – ново начално състояние с ε-преходи към двата автомата
- a* (звезда) – нови ε-преходи за образуване на цикъл
- Новосъздаденият автомат се добавя в регистъра с автоматично генерирано ID
- Извежда ID на новия автомат

Клас SelectCommand.java

```

public class SelectCommand implements Command {
    private final AutomatonManager manager;
    private final int id;
}

```

```

public SelectCommand(AutomatonManager manager, int id) {
    this.manager = manager;
    this.id = id;
}

@Override
public void execute() {
    manager.selectAutomaton(id);
}
}

```

Конкретна команда за смяна на текущо отворения автомат с друг автомат по ID. Приема един аргумент – ID на автомата, който искаме да стане текущ.

Член-данни:

- AutomatonManager manager – мениджър за автоматите
- int id – идентификатор на автомата, който ще стане текущ

Ключови методи:

- execute() – изпълнява командата (сменя текущия автомат)

Как работи:

- Командата приема един аргумент – ID на автомат
- Извиква метода selectAutomaton(id) на AutomatonManager
- Методът проверява:
 - Ако съществува автомат с подаденото ID – прави го текущ
 - Ако не съществува – извежда съобщение "No automaton with ID X"
- След смяната, текущият файл се занулява (currentFilePath = null), защото новият

автомат може да няма свързан файл

Клас ExitCommand.java

```

public class ExitCommand implements Command {
    @Override
    public void execute() {
        System.out.println("Излизане от програмата...");
        System.exit(0);
    }
}

```

Конкретна команда за излизане от програмата. Няма аргументи.

Член-данни:

- Няма – командата не изисква зависимости

Ключови методи:

- `execute()` – изпълнява командата (спира изпълнението на програмата)

Как работи:

- Командата няма аргументи
- Извежда съобщение "Излизане от програмата..."
- Извиква `System.exit(0)`, което прекратява Java виртуалната машина
- Кодът 0 означава нормално завършване (без грешка)

Клас `HelpCommand.java`

```
public class HelpCommand implements Command {  
    @Override  
    public void execute() {  
        HelpPrinter.print();  
    }  
}
```

Конкретна команда за показване на помощното меню с всички поддържани команди и техните описания. Няма аргументи.

Член-данни:

- Няма – командата не изисква зависимости

Ключови методи:

- `execute()` – изпълнява командата (извежда помощното меню)

Как работи:

- Командата няма аргументи
- Извиква статичния метод `print()` на `HelpPrinter`
- `HelpPrinter` обхожда всички стойности на изброения тип `CommandType` и

форматира командите в две колони:

- Лява колона: името на командата (с главни букви)
- Дясна колона: кратко описание на командата
- След списъка с команди се извеждат и няколко примера за употреба

Клас `Main.java`

```
public class Main {  
    public static void main(String[] args) {
```

```
CLI cli = new CLI();
cli.start();
}
}
```

Това е входната точка на програмата. Той съдържа само метода `main()`, който:

- Създава инстанция на CLI
- Извиква метода `start()`, който стартира командния интерфейс

Член-данни: няма

Ключови методи:

- `main(String[] args)` – стартира програмата

4.1. Тестване

1.Отваряне на файл с автомат

```
> open test.json
Successfully opened test.json (ID: 1)
```

2.Разпознаване на дума

```
> recognize ab
Думата "ab" се приема
```

3. Проверка за детерминираност

```
> deterministic
Автоматът е ДЕТЕРМИНИРАН
```

4. Детерминиране на автомат

```
> determinize
Детерминираният автомат е създаден с ID: 2
```

5.Обединение на два автомата

```
> union 1
Обединението е създадено с ID: 3
```

6. Конкатенация на два автомата

```
> concat 2
Конкатенацията е създадена с ID: 4
```

7. Позитивна обвивка (звезда)

```
> star
Звездата на автомата е създадена с ID: 5
```

8. Създаване на автомат от регулярен израз

```
> reg (a/b)*
Автоматът от регулярен израз е създаден с ID: 6
```

9. Проверка за празен език

```
> empty
Езикът на автомата е НЕ Е ПРАЗЕН
```

10. Записване на резултата

```
> saveas result.json
Successfully saved result.json
```

11. Отваряне на “help” командите

```
> help

NFA COMMAND LINE INTERFACE
=====
Usage: <COMMAND> [arguments]

Available commands:

HELP           Displays this help message
EXIT           Exits the program
OPEN           Opens or creates an automaton from a file
CLOSE          Closes the currently open automaton
LIST           Lists all loaded automata with their IDs
PRINT          Prints all transitions of the current automaton
SAVE           Saves the current automaton to the opened file
SAVEAS         Saves the current automaton to a new file
RECOGNIZE      Checks if a word is accepted by the automaton
DETERMINISTIC  Checks if the automaton is deterministic
DETERMINIZE    Converts NFA to DFA (subset construction)
EMPTY          Checks if the language of the automaton is empty
UNION          Creates union of current automaton with another automaton
CONCAT         Creates concatenation of current automaton with another automaton
STAR           Creates Kleene star of the current automaton
SELECT         Switches current automaton to the specified ID
REG            Creates an automaton from a regular expression

Examples:
> OPEN test.json
> RECOGNIZE ab
> UNION 1
> REG a|b*
```

Глава 5. Заключение

5.1. Обобщение на изпълнението на началните цели

Всички изисквания на проекта са изпълнени успешно. Разработената програма поддържа пълен набор от команди за работа с недетерминирани крайни автомати (NFA) с ϵ -преходи:

- Основни операции: open, close, save, saveas, help, exit
- Специфични операции: list, print, recognize, deterministic, determinize, empty, union, concat, star, reg, finite, select

Всеки автомат, зареден от файл или създаден чрез операция, получава уникален идентификатор (ID), който се използва за позоваване в командите. Програмата сериализира автоматите в JSON формат чрез библиотеката Gson.

Архитектурата на проекта следва принципите на обектно-ориентираното програмиране – използвани са Command Pattern за командите, Singleton Pattern за управление на автоматите, а кодът е разделен на отделни пакети (cli, model, io) за по-добра организация.

5.2. Насоки за бъдещо развитие и усъвършенстване

Въпреки че проектът покрива всички функционални изисквания, съществуват възможности за бъдещо развитие:

1. Графичен потребителски интерфейс (GUI) – добавяне на визуална среда за работа с автоматите, която да показва графично състоянията и преходите.
2. Оптимизация на алгоритмите – подобряване на производителността при детерминиране на големи автомати (subset construction).
3. Разширяване на регулярните изрази – поддръжка на допълнителни оператори като + (едно или повече повторения) и ? (нула или едно повторение).
4. Експорт на диаграми – възможност за генериране на визуално представяне на автомата във формат PNG или SVG.
5. Тестване – добавяне на автоматизирани unit тестове за всички операции.